

PORTFOLIO

TABLE OF CONTENTS

01

Introduction

02

Projects



ByungJoo Chae

+82 10-28030965|
wpdhkd1642@gmail.com



www.linkedin.com/in/byungjoo-chaebaa661198



Experience

**2025.01
- 2024.03**

DexterStudios
R&D Department
ML Engineer

- developing and deploying AI solutions needed by the company.



Education

**2024.02
- 2022.03**

- Chungnam National University
Master's degree
- Lab: CVIP Lab
- Professor: Prof. Donghyeon Cho

**2022.02
- 2016.03**

- Chungnam National University
Bachelor degree

Project List

Project title	Keywords	Date	Role
De-aging for VFX	Diffusion, De-aging, Video Consistency	2024.05 - 2025.01	Developer
Light Weight Ultra Style transfer Model	LightWeight Model, Ultra Style transfer	2023.12 - 2024.02	Developer
Patch-based Painterly Harmonization for High Resolution Images	Painterly Dataset, Image Harmonization	2023.05 - 2023.12	1 st Author (graduation thesis)
Development of Deep Real Technology	Harmonization, Synthetic Dataset	2022.06 - 2023.07	Developer
Online Learning for Reference-Based Super-Resolution	Online learning, Super Resolution, Reference Image	2021.06 - 2022.03	1 st Author (MDPI, Electronics)

Projects – De-aging for VFX



Introduction

- Our goal is to provide de-aging solutions for dramas and films to make actors appear younger.



Duration / People

- 2024.04 ~ Now
- One Supervisor , One Developer



Skill

- Framework : Pytorch
- Language : Python



Contribution

Role : Developer

Detail :

Version 1

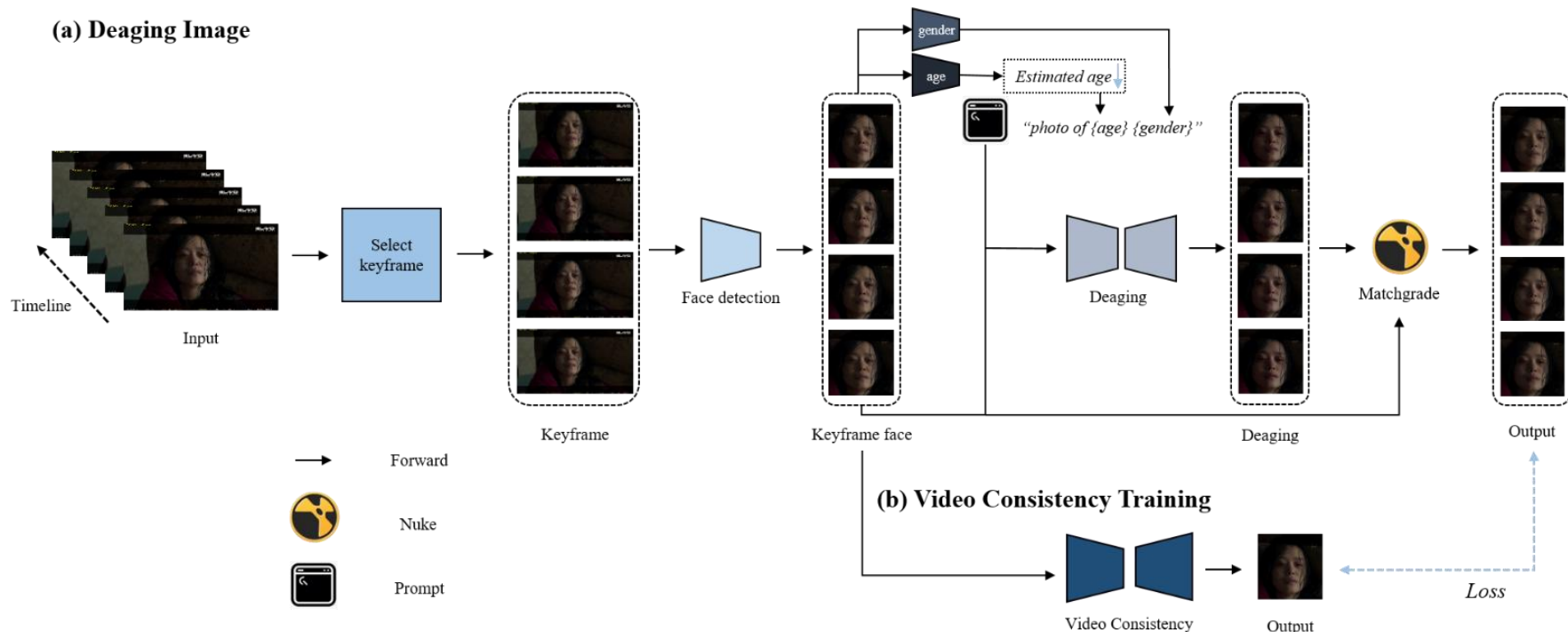
- Conducted research on de-aging models using diffusion techniques and expand the age range for de-aging by fine-tuning the model with East Asian facial datasets.
- Developed a video de-aging pipeline that ensures consistent and efficient de-aging effects across entire frames.

Version 2

- To improve the speed of the Version 1 De-aging Image model, we utilized the InstructPix2Pix model, reducing inference time by approximately 90%.

Pipeline

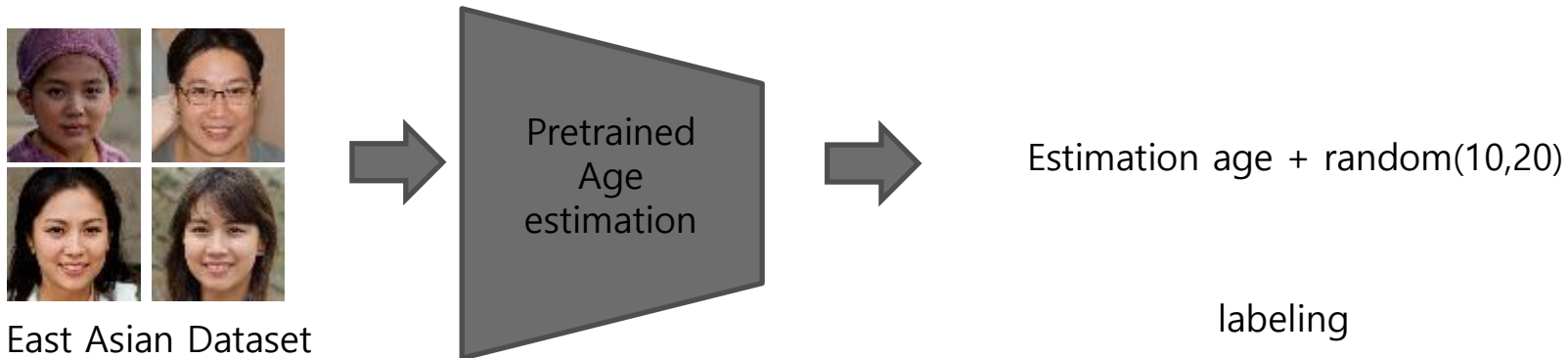
(a) Deaging Image



- The pipeline consists of a de-aging image pipeline created using the Fading diffusion model and a video consistency pipeline that applies de-aged images to videos.
- In the original Fading model, age estimation was trained on a dataset of Western individuals, often predicting the age of East Asian faces as too young. This reduced the range of possible de-aging or, in some cases, resulted in creating images of children

Dataset labeling

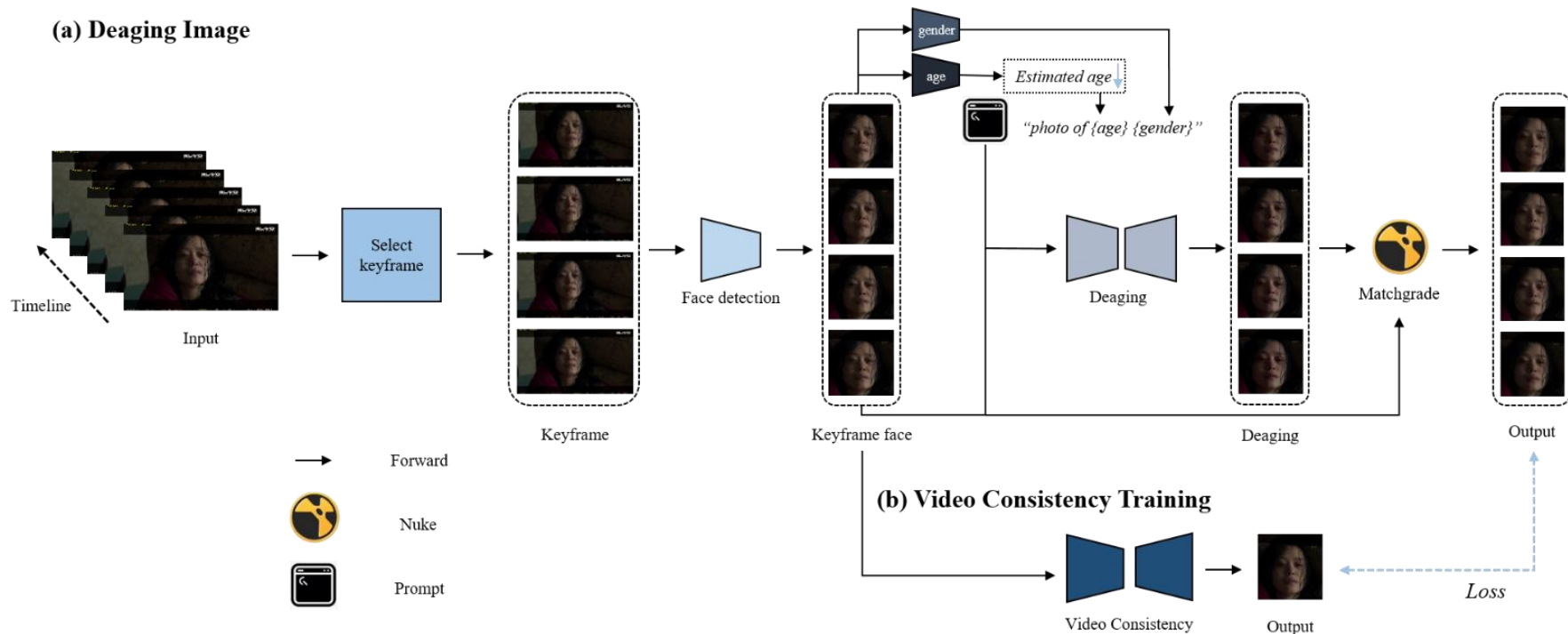
Dataset labeling



- After obtaining an East Asian dataset from Kaggle, I labeled the data by randomly adding 10 to 20 years to the labeling values generated by the pretrained age estimation model.
- The existing age estimation model tends to predict East Asians as being 10 to 20 years younger than their actual age. (Example: For Jun Ji-hyun, the existing age estimation predicts her as being in her early 20s, while her actual age is in her mid-40s.)

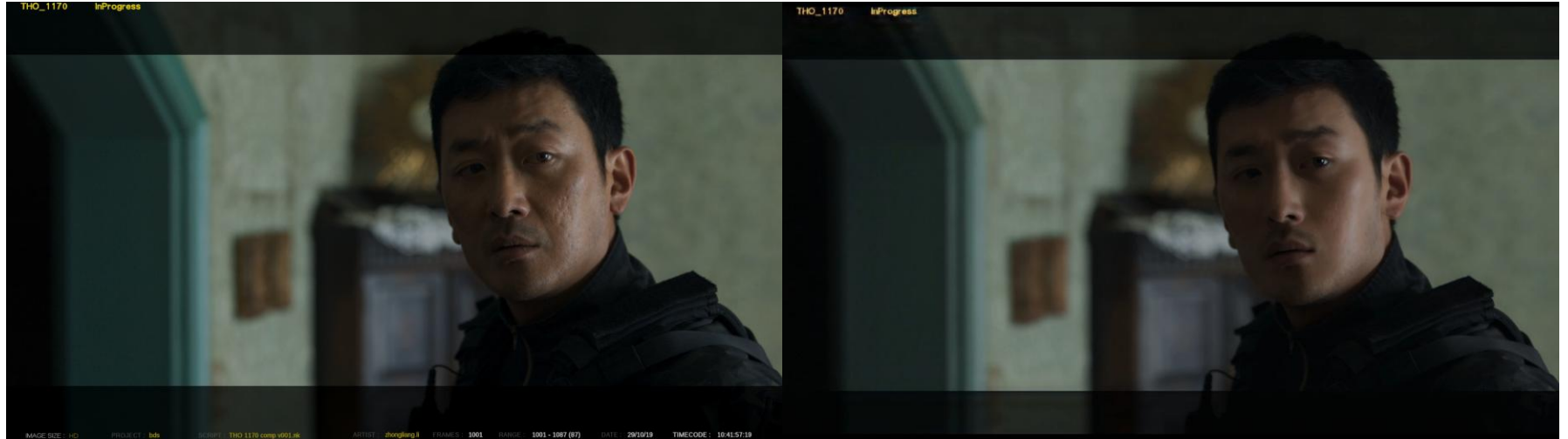
Video Consistency

(a) Deaging Image



- The video consistency model is trained by using the original image as input and the de-aged image as the ground truth (GT).
- Since overfitting is applied to a specific individual, it is possible to produce de-aged results for the same individual within a single scene.

Result



- The de-aging results show noticeable improvements in skin clarity.
- Consistency is also well maintained.

Deployment and Serving

```
FROM deaging:1.1v
RUN mkdir -p /home/dexter
COPY . /home/dexter
WORKDIR /home/dexter
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

Dockerfile



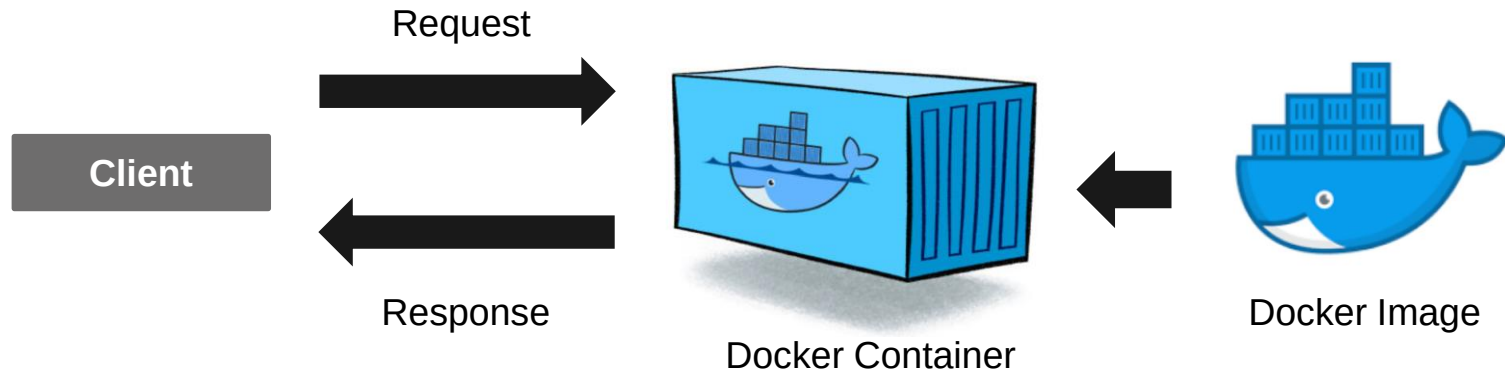
Code folder



Docker
Image

- A Dockerfile was used to build a Docker image, which was then deployed as a server to enable the use of the de-aging solution.
- The Docker image was configured with Redis, Celery, and FastAPI to set up the server environment.

Deployment and Serving



- The system was configured to operate such that the client sends requests to the server using httpx, and the server responds accordingly.
- The client runs in the Nuke environment and is packaged into a Docker image, which is then deployed on each user's local desktop equipped with a GPU.
→ This approach was deemed more efficient than setting up a dedicated server computer.

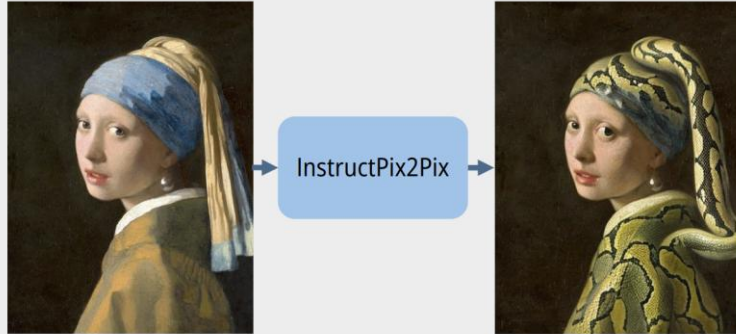
De-aging Image 0.2v

Purpose : The inference speed of the 0.1v De-aging Image model takes 90 seconds per image, so the goal is to improve its speed.

Instruction-following Diffusion Model

(d) Inference on real images:

"turn her into a snake lady"



Model

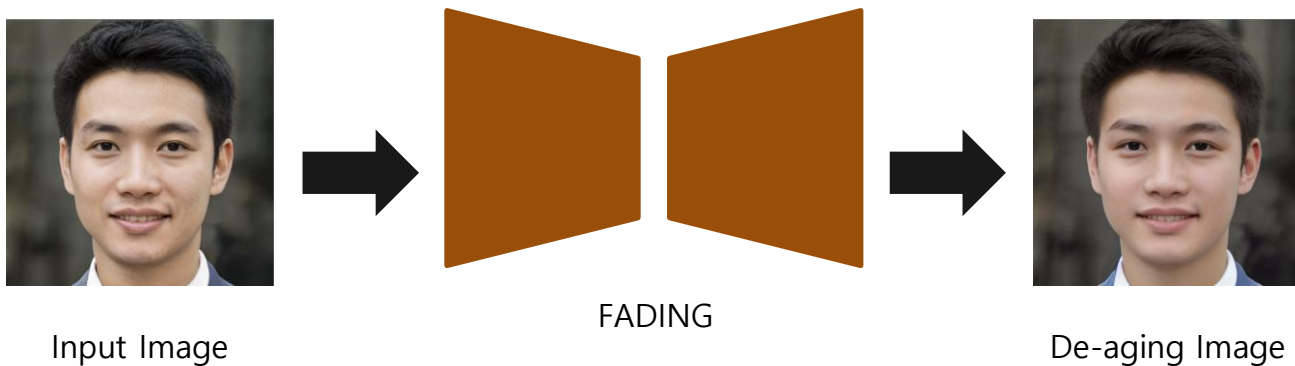


Dataset

- Choice of InstructPix2Pix Model: Its prompt-to-prompt approach ensures faster inference speed.
- Dataset: East Asian Dataset in kaggle

De-aging Image 0.2v

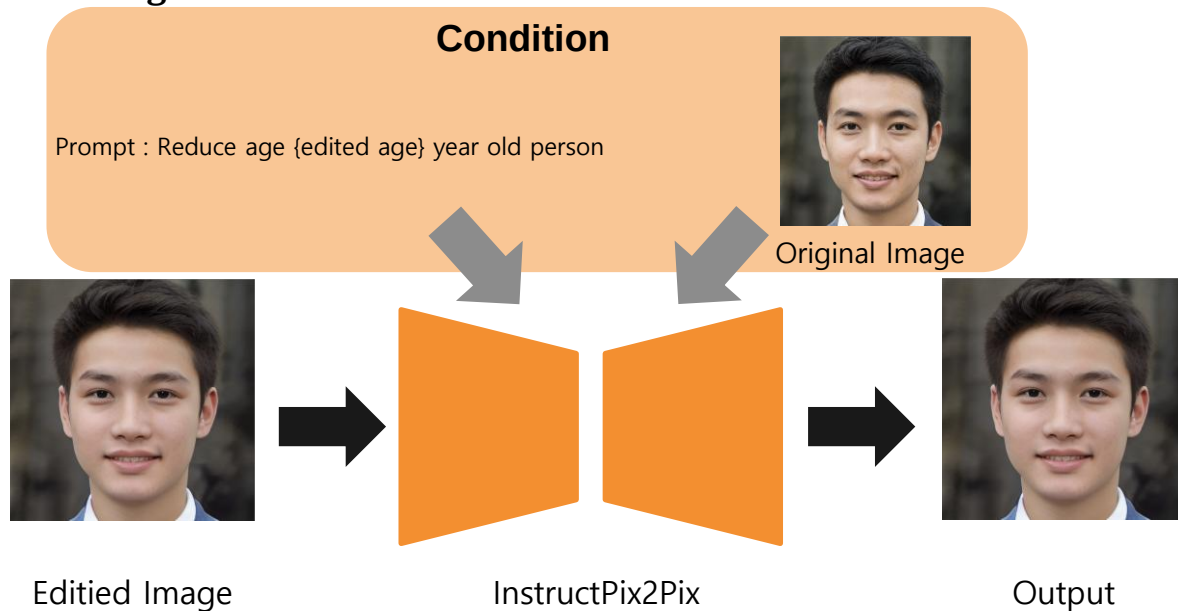
Data Preparation



- Using the existing De-aging model, input images are de-aged to create data pairs for training.
- The original age is estimated, and based on the age group (50s, 40s, 30s, 20s, or under 20s), different age ranges are applied to generate the dataset.
- The data generation criteria involve randomly applying de-aging to individuals aged 20 and above.

De-aging Image 0.2v

Fine-tuning



- Using the existing De-aging model, input images are de-aged to create data pairs for training.
- The data generation criteria involve randomly applying de-aging to individuals aged 20 and above.

Projects – Light Weight Ultra Style Transfer Model



Introduction

- The goal is to develop a lightweight model capable of ultra-resolution style transfer.



Duration / People

- 2023.12 ~ 2024.02
- One Developer



Skill

- Framework : Pytorch
- Language : Python



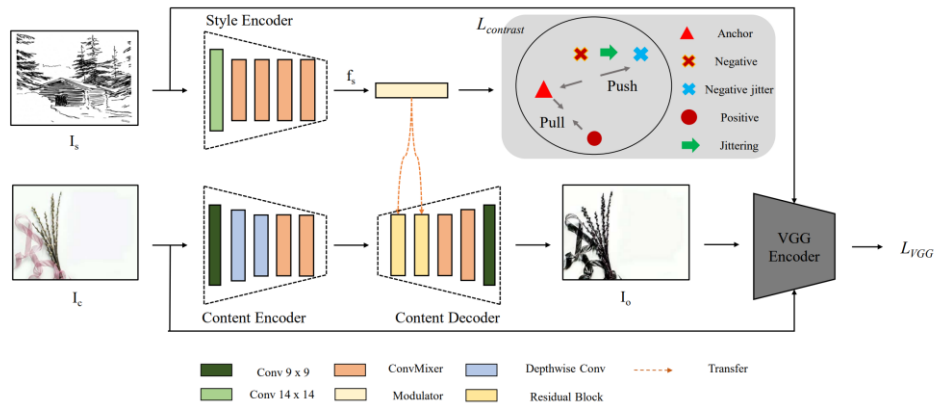
Contribution

Role : Developer

Detail :

- The overall model structure was designed to be lightweight using a module called ConvMixer.
- A learnable module was utilized to convey style information by leveraging both global and local information.

Pipeline



Overall Pipeline

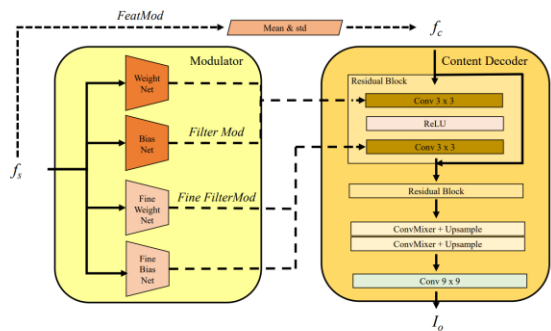
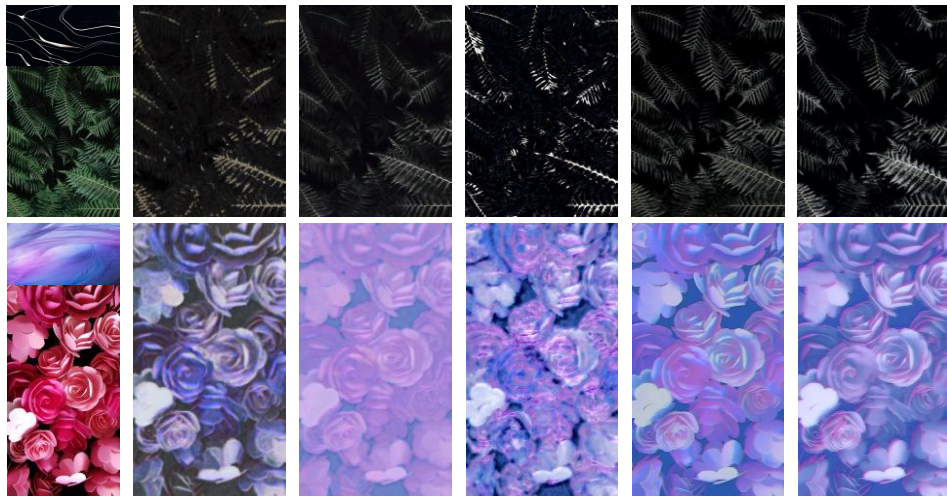


FIGURE 4. Details of our Triple Modulator

- A lightweight backbone was created using the ConvMixer module.
- To amplify the feature information obtained from the style encoder, the Triple Modulator method was applied.
- When generating negative samples, jittering was used to address class imbalance in the style dataset, specifically the WikiArt dataset.

Results



Input

AdaIN-U

LST-U

Collaborative
-Distill

MicroAST

Ours

Method	Params/ 10^6	Storage/MB	GFLOPs	Time/sec	Content Loss	Style Loss
AdaIN-U [2]	7.011	94.100	5841.9	3.59	2.086	1.209
LST-U [2]	12.167	48.600	6152.1	3.61	1.920	0.949
Collab-Distill [21]	2.146	9.659	1338.9	8.03	3.363	0.295
MicroAST [24]	0.472	1.857	374.9	0.62	2.120	0.584
Ours	0.306	1.239	226.2	0.56	1.990	0.579

- An Ultra-Style Transfer model was developed, reducing parameters and GFLOPs by 30% compared to the lightweight MicroAST model while effectively delivering style information and preserving content.

Projects – Patch-based Painterly Harmonization for High Resolution Images



Introduction

- The goal is to build a dataset for Painterly Harmonization training and improve its performance.



Duration / People

- 2023.05 ~ 2023.12
- One Developer



Skill

- Framework : Pytorch
- Language : Python



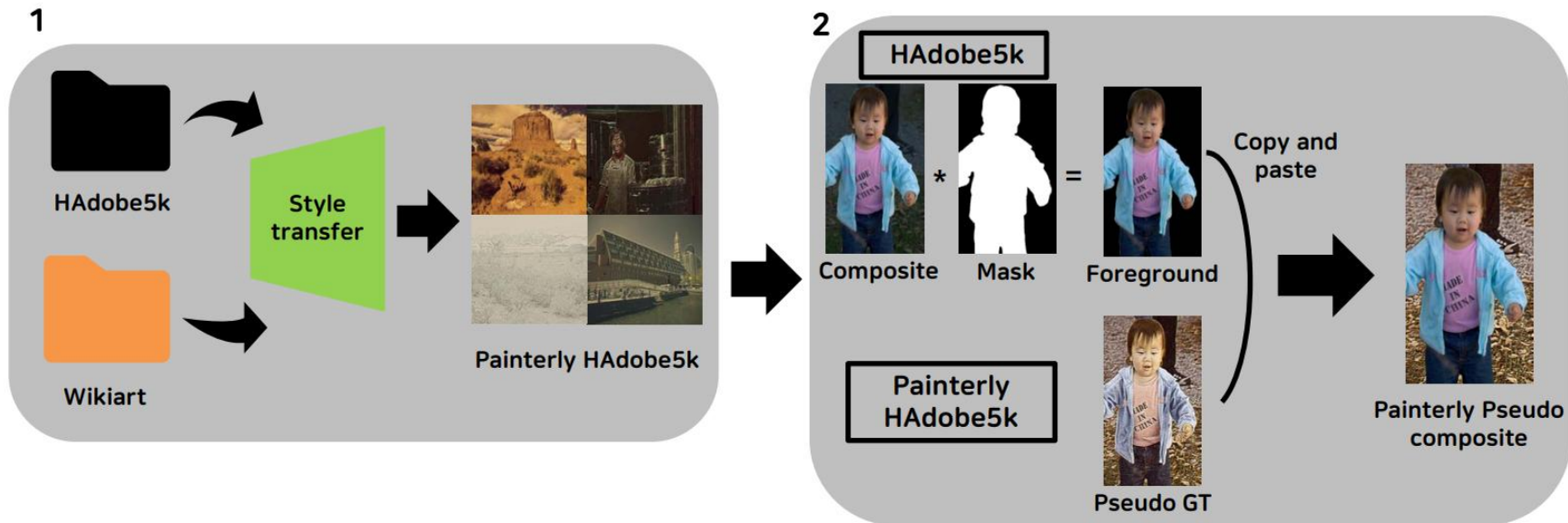
Contribution

Role : Developer

Detail :

- Built a dataset for Painterly Harmonization.
- Enhanced performance in patch-based image harmonization within the Painterly Harmonization environment.

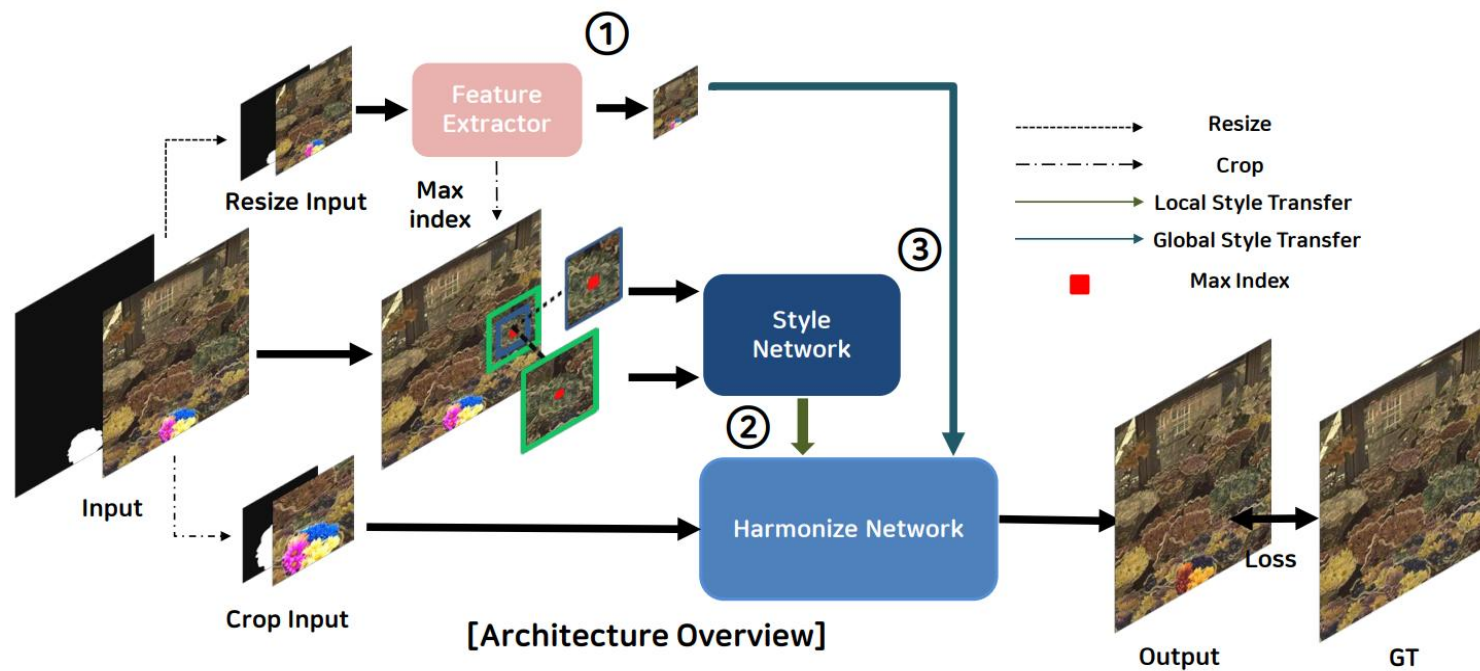
Data Generation



[Data Generate Overview]

- Due to the absence of a high-resolution painterly dataset, a new dataset was created.
- It was generated using the HAdobe5k benchmark dataset for image harmonization and the WikiArt dataset for style.

Architecture

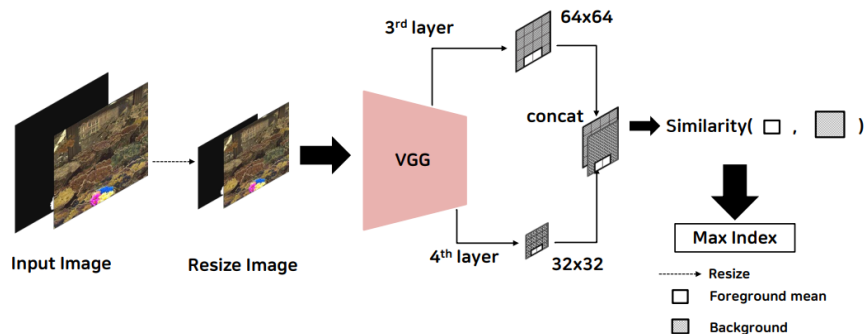


- HRNet-idih, related to high-resolution images, was used as the base model.

Model

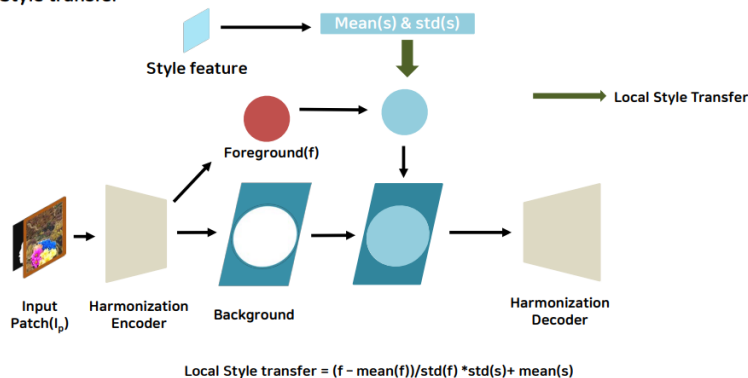
② Patch-wise Method (Feature Extractor)

► Style patch selection use VGG Network to use semantic feature



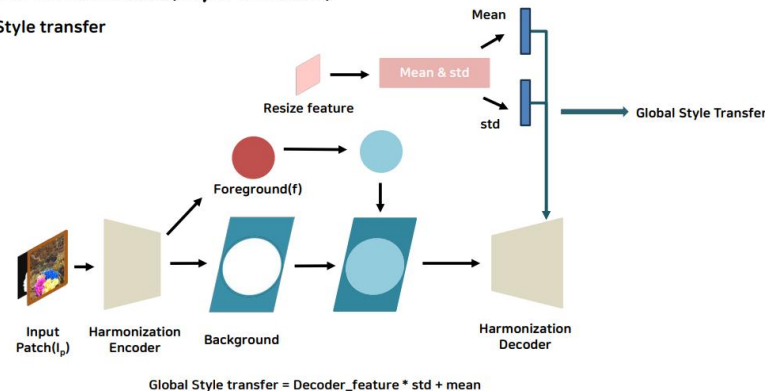
② Patch-wise Method(Style Transfer)

Local Style transfer



Patch-wise Method(Style Transfer)

Global Style transfer



- The process involves finding patches with similar characteristics to the foreground to blend with the surroundings and transfer the style.
- Both global and local information are conveyed to achieve this.

Results

Quantitative Results

1024 x 1024

Model	PSNR	fMSE	MSE
Composite Image	24.12	4360.57	674.27
Idih ^[5]	28.21	1487.14	252.86
Is ² am ^[5]	30.71	849.09	135.93
HRNET-Idih ^[5]	30.76	809.16	137.60
Ours	31.11	725.79	127.97

Qualitative Results

1024 x 1024



Composite

Idih

Is²am

HRNet-Idih

Ours

GT

- Developed a High-Resolution Painterly Dataset.
- Enhanced performance by utilizing both local and global information.
- Improved patch-based image harmonization performance in the Painterly Harmonization environment.

Projects – Development of Deep Real Technology



Introduction

- The goal is to acquire a dataset suitable for harmonization, capable of synthesizing the foreground into the background in a synthetic environment, and train a model using it.



Duration / People

- 2022.07 ~ 2023.06
- One Developer, One Company



Skill

- Framework : Pytorch
- Language : Python



Contribution

Role : Developer

Detail :

- Proposed a method for data generation.
- Automated the creation of a synthetic harmonization dataset.

Data Generate

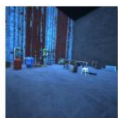
1. Data Generate



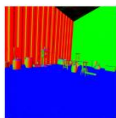
VdjrnDataGen



Basecolor



MainView



Normal



ObjId

Data Generation

Synthetic Dataset

2. Mask Generate



ObjID



Mask 이미지 예시

3. Composite Image Generate



BaseColor



Mask

*

=



Foreground

Jittering

→



Target Image

→

Copy and Paste

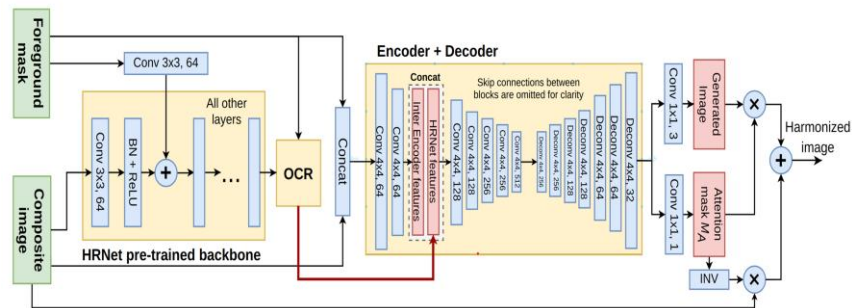


Composite Image

- Automated the creation of a synthetic harmonization dataset.
- Developed a process to obtain 27,157 composite images, masks, and ground truth (GT) required for training from the raw dataset available in the Unreal environment.

Pipeline

Harmonization Model



Image



Input

Output

GT

Mask

Metric

Model	MSE	fMSE	PSNR
Base Model	162.81	2145.53	27.36
Fine-tuning	49.00	549.05	32.43

- Trained the harmonization model using HRNet-idih with 26,157 train dataset and evaluate the 1,000 evaluate dataset.
- As a result, the masked areas were successfully blended with the surrounding environment.

Projects – Online Learning for Reference-Based Super-Resolution



Introduction

- The goal is to improve the performance of the SR model using reference images.



Duration

- 2023.05 ~ 2023.12



Skill

- Framework : Pytorch
- Language : Python



Contribution

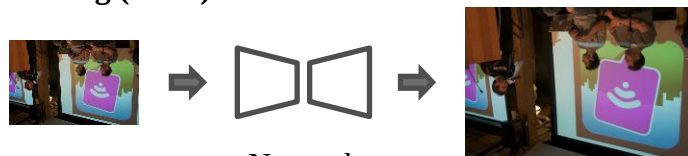
Role : 1st Author

Detail :

- Proposed an online learning method for Reference-based Super Resolution to train with diverse data pairs.
- The method is simple yet effective and can be seamlessly integrated into both SISR and RefSR models.
- It demonstrates consistent performance improvements regardless of the degree of similarity between the reference image and the input image.

Pipeline

Online learning (SISR)

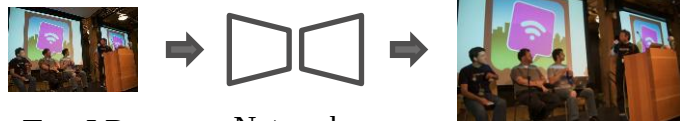


Data Pair(X)

Network

Data Pair(Y)

Inference

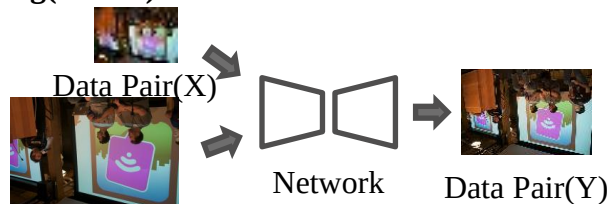


Test LR

Network

SR

Online learning(RefSR)



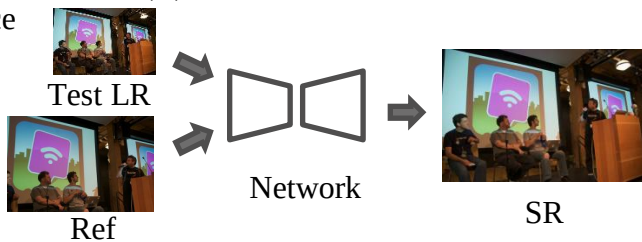
Data Pair(X)

Network

Data Pair(Y)

Data Pair(R)

Inference



Test LR

Network

SR

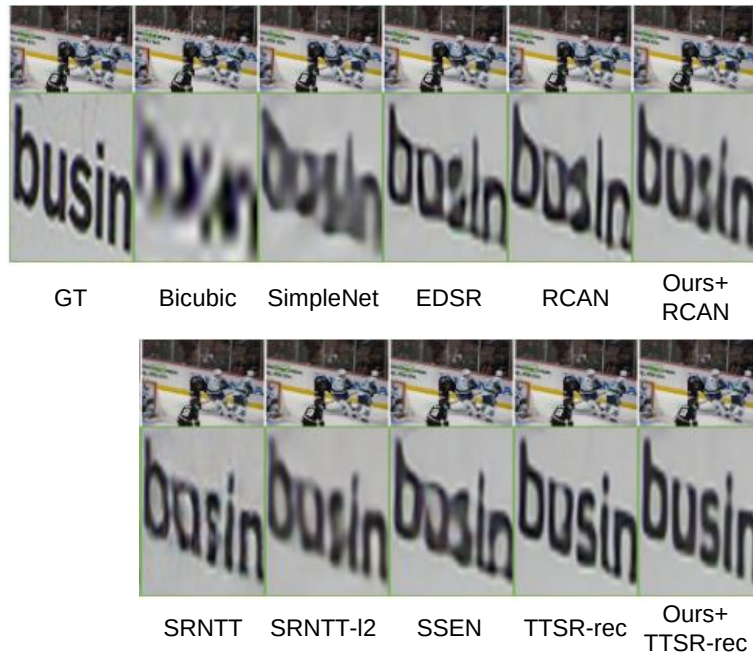
Model	SISR			RefSR			
Data Pair	D_s^{LR}	D_s^{Pse}	D_s^{Ref}	D_r^{LR}	D_r^{Pse}	D_r^{Ref1}	D_r^{Ref2}
X	$I^{LR} \downarrow$	$\bar{I}^{HR} \downarrow$	$I^{Ref} \downarrow$	$I^{LR} \downarrow$	$\bar{I}^{HR} \downarrow$	$I^{Ref} \downarrow$	$I^{Ref} \downarrow$
R	-	-	-	I^{Ref}	I^{Ref}	I^{LR}	$\bar{I}^{HR}$
Y	I^{LR}	$\bar{I}^{HR}$	I^{Ref}	I^{LR}	$\bar{I}^{HR}$	I^{Ref}	I^{Ref}

- Conducted experiments on various data pairs, including reference images.
- Achieved good performance in both SISR and RefSR models without requiring any additional modules.

Results

Model	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
RCAN	Pre-trained	26.243	0.774	0.2906
Ours + RCAN	D_s^{Ref}	26.703	0.782	0.2856
	$D_s^{LR} + D_s^{Ref}$	26.810	0.785	0.2796
	$D_s^{Pse} + D_s^{Ref}$	26.634	0.781	0.2887
	$D_s^{LR} + D_s^{Pse} + D_s^{Ref}$	26.681	0.782	0.2862

Model	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
SRNTT	Pre-trained	25.17	0.734	0.2099
SRNTT- l_2	Pre-trained	26.06	0.765	0.2758
SSEN	Pre-trained	26.412	0.776	0.2915
Ours + SSEN	D_r^{LR}	26.112	0.764	0.2884
	D_r^{Pse}	26.527	0.777	0.2930
	D_r^{Ref1}	26.276	0.770	0.2895
	D_r^{Ref2}	26.675	0.780	0.2874
	$D_r^{LR} + D_r^{Ref1}$	26.257	0.769	0.2946
	$D_r^{Pse} + D_r^{Ref2}$	26.568	0.777	0.2924
TTSR-rec	Pre-trained	27.039	0.799	0.2653
Ours + TTSR-rec	D_r^{LR}	26.812	0.788	0.2545
	D_r^{Pse}	27.337	0.801	0.2663
	D_r^{Ref1}	26.986	0.794	0.2614
	D_r^{Ref2}	27.383	0.802	0.2578
	$D_r^{LR} + D_r^{Ref1}$	26.900	0.790	0.2529
	$D_r^{Pse} + D_r^{Ref2}$	27.400	0.801	0.2663



Results

Model	Kernel	Blind	Method	PSNR	SSIM	LPIPS	Model	Kernel	Blind	Method	PSNR	SSIM	LPIPS
Ours+RCAN [7]	$g_{0.2}^d$	-	Pre-trained	17.938	0.497	0.4111	Ours+TTSR-rec [18]	$g_{0.2}^d$	-	Pre-trained	18.415	0.524	0.4039
		Non-blind	$D_s^{LR} + D_s^{Ref}$	24.532	0.737	0.2910			Non-blind	D_r^{Ref1}	23.489	0.688	0.3423
	$g_{2.0}^d$	-	Pre-trained	21.131	0.597	0.3335		$g_{2.0}^d$	-	Pre-trained	21.211	0.609	0.3127
		Non-blind	$D_s^{LR} + D_s^{Ref}$	26.545	0.783	0.2586			Non-blind	D_r^{Ref1}	25.911	0.760	0.2647
	g_{ani}^d	-	Pre-trained	21.198	0.587	0.3609		g_{ani}^d	-	Pre-trained	21.199	0.596	0.3367
		Non-blind	$D_s^{LR} + D_s^{Ref}$	26.414	0.775	0.2679			Non-blind	D_r^{Ref1}	25.512	0.741	0.2841
	$g_{1.3}^b$	-	Pre-trained	25.484	0.738	0.3314		$g_{1.3}^b$	-	Pre-trained	26.147	0.767	0.2912
		Non-blind	$D_s^{LR} + D_s^{Ref}$	26.909	0.790	0.2597			Non-blind	D_r^{Ref1}	26.599	0.781	0.2471
	-	-	Pre-trained	21.798	0.606	0.3914		-	-	Pre-trained	21.820	0.615	0.3603
		Blind	$D_s^{LR} + D_s^{Ref}$	24.277	0.692	0.3480			Blind	D_r^{Ref1}	23.928	0.672	0.3535

- Achieved better results by using reference images, even with various isotropic and anisotropic kernels.



THANKS!

Do you have any questions?

wpdhkd1642@gmail.com

+82 10-2803-0965